Day2_AWK

Write an AWK program for each question. Ignore the header row where appropriate.
1. Print the source IP and event from every record.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NR > 1 {print $2, $3}' security_log.csv
192.168.1.10 Login_Success
192.168.1.25 Login_Failed
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
192.168.1.30 Login_Success
```

2. Display the time, source IP, and attempts in aligned columns using printf.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NR > 1 {printf "%-10s %-16s %s\n", $1, $2, $5}' security_log.csv
09:10      192.168.1.10     1
09:12      192.168.1.25     3
09:15      10.0.0.8         12
09:18      192.168.1.25     7
09:22      10.0.0.12        1
09:25      192.168.1.30     1
```

3. Store the source IP and event in variables, then print them in a sentence.

Script:

```
 GNU nano 8.7.1                                  3.awk *
NR > 1 {
    ip = $2
    event = $3
    print "Source IP " ip " generated event " event
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 3.awk security_log.csv
Source IP 192.168.1.10 generated event Login_Success
Source IP 192.168.1.25 generated event Login_Failed
Source IP 10.0.0.8 generated event Port_Scan
Source IP 192.168.1.25 generated event Login_Failed
Source IP 10.0.0.12 generated event Malware_Alert
Source IP 192.168.1.30 generated event Login_Success
```

4. Calculate the processing time for each record if one attempt takes 2 seconds.

Script:

```
 GNU nano 8.7.1                                  4.awk *
NR > 1 {
    time = $5 * 2
    print $2, $3, time " seconds"
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 4.awk security_log.csv
192.168.1.10 Login_Success 2 seconds
192.168.1.25 Login_Failed 6 seconds
10.0.0.8 Port_Scan 24 seconds
192.168.1.25 Login_Failed 14 seconds
10.0.0.12 Malware_Alert 2 seconds
192.168.1.30 Login_Success 2 seconds
```

5. Calculate the total number of attempts in the file.

Script:

```
 GNU nano 8.7.1                                                 5.awk *
NR > 1 {total += $5}
END {print "Total attempts:", total}'
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 5.awk security_log.csv
Total attempts: 25
```

6. Calculate the average number of attempts per record.

Script:

```
 GNU nano 8.7.1                                                 6.awk *
awk 'NR > 1 {
    total += $5
    count++
}
END {
    print "Average attempts:", total / count
}' security_log.csv
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ ./6.awk
Average attempts: 4.16667
```

7. Count the number of Login_Failed records.

Script:

```
 GNU nano 8.7.1                                                 7.awk *
awk 'NR > 1 && $3 == "Login_Failed" {count++}
END {print "Login Failed records:", count+0}' security_log.csv
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ ./7.awk
Login Failed records: 2
```

8. Start with a review quota of 10 and decrease it by one for every HIGH or CRITICAL record. Print the remaining quota.

Script:

```
 GNU nano 8.7.1                                          8.awk *
BEGIN {
    quota = 10
}

NR > 1 {
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        quota--
    }
}

END {
    print "Remaining quota:", quota
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 8.awk security_log.csv
Remaining quota: 7
```

9. Replace Port_Scan with Network_Scan in the event field and print every record in CSV format.

Script:

```
 GNU nano 8.7.1                                          9.awk *
BEGIN {
    FS = "[[:space:]]+"
    OFS = ","
}

NR == 1 {
    next
}

{
    if ($3 == "Port_Scan") {
        $3 = "Network_Scan"
    }

    print $1, $2, $3, $4, $5
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 9.awk security_log.csv
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,3
09:15,10.0.0.8,Network_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

10. Add one to the attempts field of each Login_Failed record and print every record in CSV format.

Script:

```
 GNU nano 8.7.1                                                  10.awk *
BEGIN {
    FS = "[[:space:]]+"
    OFS = ","
}

NR == 1 {
    next
}

{
    if ($3 == "Login_Failed") {
        $5 = $5 + 1
    }

    print $1, $2, $3, $4, $5
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 10.awk security_log.csv
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Login_Failed,MEDIUM,4
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Login_Failed,HIGH,8
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

11. Print the source IP and event when severity is HIGH and attempts are greater than 5.

Script:

```
 GNU nano 8.7.1                                                 11.awk *
NR > 1 {
    if ($4 == "HIGH" && $5 > 5) {
        print $2, $3
    }
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 11.awk security_log.csv
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
```

12. Print the source IP and event when severity is HIGH or CRITICAL.

Script:

```
 GNU nano 8.7.1                                                 12.awk *
NR > 1 {
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        print $2, $3
    }
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 12.awk security_log.csv
10.0.0.8 Port_Scan
192.168.1.25 Login_Failed
10.0.0.12 Malware_Alert
```

13. Use the conditional operator to label each record Repeated if attempts exceed 1; otherwise label it Single.

Script:

```
 GNU nano 8.7.1                                                 13.awk *
NR > 1 {
    label = ($5 > 1) ? "Repeated" : "Single"
    print $2, $3, label
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 13.awk security_log.csv
192.168.1.10 Login_Success Single
192.168.1.25 Login_Failed Repeated
10.0.0.8 Port_Scan Repeated
192.168.1.25 Login_Failed Repeated
10.0.0.12 Malware_Alert Single
192.168.1.30 Login_Success Single
```

14. Use the conditional operator to label each record Investigate if attempts are at least 5; otherwise label it Routine.

Script:

```
 GNU nano 8.7.1                                                  14.awk *
NR > 1 {
    label = ($5 >= 5) ? "Investigate" : "Routine"
    print $2, $5, label
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 14.awk security_log.csv
192.168.1.10 1 Routine
192.168.1.25 3 Routine
10.0.0.8 12 Investigate
192.168.1.25 7 Investigate
10.0.0.12 1 Routine
192.168.1.30 1 Routine
```

15. Use if–else to print Review required for HIGH or CRITICAL records and Routine review for all other records.

Script:

```
 GNU nano 8.7.1                                                  15.awk *
NR > 1 {
    if ($4 == "HIGH" || $4 == "CRITICAL") {
        print $2, "Review required"
    }
    else {
        print $2, "Routine review"
    }
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 15.awk security_log.csv
192.168.1.10 Routine review
192.168.1.25 Routine review
10.0.0.8 Review required
192.168.1.25 Review required
10.0.0.12 Review required
192.168.1.30 Routine review
```

16. Use if–else if–else to classify attempts as High activity (10 or more), Moderate activity (5–9), or Low activity (fewer than 5).

Script:

```
  GNU nano 8.7.1                                                16.awk *
NR > 1 {
    if ($5 >= 10) {
        activity = "High activity"
    }
    else if ($5 >= 5) {
        activity = "Moderate activity"
    }
    else {
        activity = "Low activity"
    }

    print $2, $5, activity
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 16.awk security_log.csv
192.168.1.10 1 Low activity
192.168.1.25 3 Low activity
10.0.0.8 12 High activity
192.168.1.25 7 Moderate activity
10.0.0.12 1 Low activity
192.168.1.30 1 Low activity
```

17. Use a for loop to print Check 1 through Check 5.

Script:

```
 GNU nano 8.7.1                                                 17.awk *
BEGIN {
    for (i = 1; i <= 5; i++) {
        print "Check", i
    }
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 17.awk security_log.csv
Check 1
Check 2
Check 3
Check 4
Check 5
```

18. Use a while loop to count down from 10 to 1.

Script:

```
 GNU nano 8.7.1                                                 18.awk *
BEGIN {
    i = 10

    while (i >= 1) {
        print i
        i--
    }
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 18.awk security_log.csv
10
9
8
7
6
5
4
3
2
1
```

19. Use a do–while loop to count down from 10 to 1.

Script:

```
 GNU nano 8.7.1                                                 19.awk *
BEGIN {
    i = 10

    do {
        print i
        i--
    } while (i >= 1)
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 19.awk security_log.csv
10
9
8
7
6
5
4
3
2
1
```

20. Print each record's source IP, event, attempts, and activity classification in aligned columns. At the end, print the total records and total attempts.

Script:

```
 GNU nano 8.7.1                                                  20.awk *
BEGIN {
    printf "%-16s %-18s %-10s %-20s\n", \
           "Source IP", "Event", "Attempts", "Activity"
}

NR > 1 {
    if ($5 >= 10) {
        activity = "High activity"
    }
    else if ($5 >= 5) {
        activity = "Moderate activity"
    }
    else {
        activity = "Low activity"
    }

    printf "%-16s %-18s %-10d %-20s\n", \
           $2, $3, $5, activity

    records++
    total += $5
}

END {
    print "Total records:", records
    print "Total attempts:", total
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 20.awk security_log.csv
Source IP        Event              Attempts   Activity
192.168.1.10     Login_Success      1          Low activity
192.168.1.25     Login_Failed       3          Low activity
10.0.0.8         Port_Scan          12         High activity
192.168.1.25     Login_Failed       7          Moderate activity
10.0.0.12        Malware_Alert      1          Low activity
192.168.1.30     Login_Success      1          Low activity
Total records: 6
Total attempts: 25
```

21.Login monitoring: A security analyst considers five or more failed attempts suspicious. Write an AWK script that prints the source IP and attempts for matching Login_Failed entries. At the end, print how many suspicious entries were found.

Script:

```
 GNU nano 8.7.1                                                21.awk *
NR > 1 {
    if ($3 == "Login_Failed" && $5 >= 5) {
        print $2, $5
        count++
    }
}

END {
    print "Suspicious entries:", count+0
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 21.awk security_log.csv
192.168.1.25 7
Suspicious entries: 1
```

21. Incident summary: Write an AWK script that counts the entries in each severity category and prints a summary in an END block.

Script:

```
 GNU nano 8.7.1                                                22.awk *
NR > 1 {
    severity[$4]++
}

END {
    print "Incident Summary"
    print "LOW:", severity["LOW"]+0
    print "MEDIUM:", severity["MEDIUM"]+0
    print "HIGH:", severity["HIGH"]+0
    print "CRITICAL:", severity["CRITICAL"]+0
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 22.awk security_log.csv
Incident Summary
LOW: 2
MEDIUM: 1
HIGH: 2
CRITICAL: 1
```

23. Processing estimate: Assume each attempt requires 2 seconds of analyst review. Calculate the review time for each entry and the total review time for the file. Use printf to present the results neatly.

Script:

```
  GNU nano 8.7.1                                                23.awk *
BEGIN {
    printf "%-10s %-16s %-10s %-15s\n", \
           "Time", "Source IP", "Attempts", "Review (sec)"
}

NR > 1 {
    review = $5 * 2

    printf "%-10s %-16s %-10d %-15d\n", \
           $1, $2, $5, review

    total += review
}

END {
    printf "Total review time: %d seconds\n", total
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 23.awk security_log.csv
Time       Source IP        Attempts   Review (sec)
09:10      192.168.1.10     1          2
09:12      192.168.1.25     3          6
09:15      10.0.0.8         12         24
09:18      192.168.1.25     7          14
09:22      10.0.0.12        1          2
09:25      192.168.1.30     1          2
Total review time: 50 seconds
```

24. Log normalization: A monitoring tool requires the event name Failed_Login instead of Login_Failed. Update the matching field and print all records as CSV. The source file should remain unchanged.

Script:

```
 GNU nano 8.7.1                                                 24.awk *
BEGIN {
    OFS = ","
}

NR == 1 {
    next
}

{
    if ($3 == "Login_Failed") {
        $3 = "Failed_Login"
    }

    print $1, $2, $3, $4, $5
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 24.awk security_log.csv
09:10,192.168.1.10,Login_Success,LOW,1
09:12,192.168.1.25,Failed_Login,MEDIUM,3
09:15,10.0.0.8,Port_Scan,HIGH,12
09:18,192.168.1.25,Failed_Login,HIGH,7
09:22,10.0.0.12,Malware_Alert,CRITICAL,1
09:25,192.168.1.30,Login_Success,LOW,1
```

25. Priority queue: Assign each entry a priority: Immediate for CRITICAL, Soon for HIGH, and Routine for all others. Print the time, source IP, event, and assigned priority.

Script:

```
 GNU nano 8.7.1                                                 25.awk *
BEGIN {
    printf "%-10s %-16s %-18s %-10s\n", \
           "Time", "Source IP", "Event", "Priority"
}

NR > 1 {
    if ($4 == "CRITICAL") {
        priority = "Immediate"
    }
    else if ($4 == "HIGH") {
        priority = "Soon"
    }
    else {
        priority = "Routine"
    }

    printf "%-10s %-16s %-18s %-10s\n", \
           $1, $2, $3, priority
}
```

Output:

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f 25.awk security_log.csv
Time       Source IP        Event              Priority
09:10      192.168.1.10     Login_Success      Routine
09:12      192.168.1.25     Login_Failed       Routine
09:15      10.0.0.8         Port_Scan          Soon
09:18      192.168.1.25     Login_Failed       Soon
09:22      10.0.0.12        Malware_Alert      Immediate
09:25      192.168.1.30     Login_Success      Routine
```